



Wormhole Rent Reclaim

Security Assessment

May 25th, 2026 — Prepared by OtterSec

Akash Gurugunti

sud0u53r.ak@osec.io

Mingxuan Yang

ymxf00l@osec.io

Table of Contents

Executive Summary	2
Overview	2
Key Findings	2
Scope	2
Vulnerabilities	3
OS-WRR-ADV-00 Backdated Legacy VAAs Bypass Retention	4
Appendices	
Vulnerability Rating Scale	5
Procedure	6

01 — Executive Summary

Overview

Wormhole foundation engaged OtterSec to assess the `rent-reclaim` program. This assessment was conducted between May 18th and May 20th, 2026. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 1 finding throughout this audit engagement.

In particular, we identified a vulnerability where the default submission time backdates all legacy Posted-VAAAs whose submission time is zero to `2026-04-10`, so after the retention period elapses, they become immediately closable regardless of their real posting time, bypassing the intended 30-day retention buffer ([OS-WRR-ADV-00](#)).

Scope

The source code was delivered to us in a Git repository at <https://github.com/wormhole-foundation/wormhole>. This audit was performed against commit `9914d9d`.

A brief description of the program is as follows:

Name	Description
rent-reclaim	Reclaims rent from intermediate accounts created by the Core bridge during message publishing and verification on SVM chains, such as Solana.

02 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-WRR-ADV-00	LOW	RESOLVED ✓	<code>DEFAULT_SUBMISSION_TIME</code> back-dates all legacy <code>PostedVAAs</code> with <code>submission_time == 0</code> to <code>2026-04-10</code> , so after the retention period elapses they become immediately closable regardless of their real posting time, bypassing the intended 30-day retention buffer.

Backdated Legacy VAAs Bypass Retention LOW

OS-WRR-ADV-00

Description

`DEFAULT_SUBMISSION_TIME` creates a problematic backdating behavior for `PostedVAA` accounts whose `submission_time` is still zero. Instead of treating the missing timestamp conservatively from the upgrade/deployment point, the handler substitutes a fixed historical timestamp, `2026-04-10`, and then applies the normal `RETENTION_PERIOD` check against that value. Once the chain time is beyond `2026-05-10`, every pre-upgrade `PostedVAA` with `submission_time == 0` is considered outside the 30-day retention window, regardless of when it was actually posted.

```
>_ solana/bridge/program/src/api/close_signature_set_and_posted_vaa.rs
```

RUST

```
/// Default submission_time for accounts created before the submission_time  
/// field was populated. Gives legacy accounts a 30-day grace period from  
/// this date (10 April 2026) instead of being immediately closable.  
const DEFAULT_SUBMISSION_TIME: u32 = 1775833078;
```

As a result, recently created but pre-upgrade VAAs may become immediately closable by any caller at deployment time. This weakens the intended retention guarantee because legacy accounts are not granted a grace period, and they are effectively backdated into expiry.

Remediation

Set the legacy fallback timestamp to the upgrade/deployment activation time.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.